

Gesture Recognition and Control

Michael Lord

Advisor: Dr. Yu-Ju Lin

April 20, 2011

TABLE OF CONTENTS

SENIOR PROJECT

INTRODUCTION.....	2
Project Background and Purpose.....	2
Project Summary.....	2
Project Timeline.....	3

RESEARCH

Similar Technologies.....	4
Image Formats.....	6
Image Comparisons.....	7
Algorithm Issues.....	8

TOUCHLESS SDK

Attempts to Alter the SDK.....	10
Advantages of Using the SDK.....	10
Disadvantages of using the SDK.....	10
Using Touchless.....	11

GESTURE ALGORITHMS

Unique gesture problems.....	12
A comparative algorithm.....	13
A relative algorithm.....	13

POTENTIAL IMPROVMENTS

Gesture Testing.....	15
Algorithm improving.....	15
Interface tweaking.....	15
Additional features.....	16
More Fun Ideas.....	16

APPENDIXES

Appendix A

 Original Webcam Capture Code

Appendix B

 Demonstration Step-by-Step instructions

 First Demonstration Code

 Final Demonstration Code

Appendix C

 Touchless SDK Documentation

 Touchless SDK Class Diagram

 Touchless SDK Maker Searching Code

INTRODUCTION

Introduction

Recent sale trends have shown that having the latest and greatest technology is not the only aspect to marketing a product. A nice user interface and an easily controlled system play a significantly large role in how well the product will sell. The relatively recent implementation of touch screens onto mobile devices has been one example of such a technology. After a discussion with Dr. Lin, we decided that it might be advantageous to research a new method of control for something much more common, the PC. Controlling the PC through the use of gestures captured by nothing more than a standard webcam could provide a potential improvement for many different situations.

Project Summary

This project aims to create a method of controlling a standard computer through the use of gestures such as waving a hand to the left to page through documents. To make the system as user friendly and cost efficient as possible, the project aims to use nothing more than a standard webcam to provide this functionality. Potential applications of the project include but are not limited to controlling a computer cursor, video game simulations, specific software navigation and further knowledge or research into the concepts of facial recognition and electronic recognition of emotions. The project can potentially lead to new and advanced ideas or improvements in many realms of computer science.

Project Background

The standard keyboard remains pretty much unchanged since its conception in the 1800s on the standard typewriter (QWERTY). Unfortunately, it is impractical to think that we can simply make a gesture for each and every key. That would be over 250 keys for the Extended American Standard Code for Information Interchange (ASCII) format. The mouse on the other hand can be limited to nothing more than a handful of buttons and an X, Y coordinate and in many situations is simpler to use.

Initially upon the decision of taking on this task as a Senior Project, Dr. Lin posed one major question: is it possible to capture an image from any generic webcam and have it easily assessable to code for manipulation and comparison. For the sake of simplicity, he had encoded a project in the Visual Basic language that initialized a webcam, turned it on and captured an image onto the Windows clipboard. Initially I had thought that it would be quite simple after this. Re-encode the program into a faster language, and use multiple images to compare and find changes or track an item's movement. As one might expect, it wasn't nearly this simple.

Project Timeline

Jan 2010	Create Webcam Access and Capture Code in C#
Feb 2010 – July 2010	Research Image Processing
Aug 2010	Touchless SDK Research
Sep 2010	Improvements to the Touchless SDK
Oct 2010 – Dec 2010	Movement Algorithms
Jan 2011	Implementation of Demonstration Code
Mar 2011	Debugging and Tweaking

RESEARCH

Research

Because CSU does not offer any particular courses designed around handling images computationally, a large portion of research had to be conducted. To make matters worse, most of the documentation found online was either oversimplified or too complicated. Ultimately most of my foundational research came from a purchased text book titled “Principles of digital Image Processing” by Wilhelm Burger and Mark J. Burge.

Similar Technologies

When this project began back in the spring of 2010, a few similar technologies had not yet been released. The most widely known technologies are related to console gaming. The Nintendo Wii was the only system released at the time of this project’s conception. Afterward the Kinect system for Microsoft’s Xbox 360 was released as well as the PlayStation Move.

The Nintendo Wii was the first pioneer of bringing wireless motion detection into the home. The system gained large amounts of popularity due to this new feature but the technology behind it is not exactly common. The Wii system includes a Wii Sensor Bar as well as a proprietary Wii remote for each player. The Wii Sensor Bar is simply a bar with two sets of 5 infrared LEDs separated by a small distance, and each Wii Remote contains infrared cameras that are designed to sense the distance and changes between the two infrared lights transmitted by the Wii Sensor Bar. The remotes then wirelessly transmit their observations back to the console via bluetooth and the games act accordingly. Unfortunately, not very many people have an abundance of infrared transmitters, high definition infrared cameras, or the wireless transmitters to transmit the observation back to their PC. In addition the Wii remotes contain accelerometers and converters that work to figure out the changes in the remote’s position without the use of the camera. These additional features make it much easier for the console to get an accurate fix on the remote’s location as well as being able to maintain control even when the Wii remote cannot see any of the infrared lights from the sensor bar (Inside).

In contrast the Kinect system for the Xbox 360 requires no remotes and advertises itself with the slogan “you are the controller”. This is much closer to the goal of this project, but was not released until more than half way through the project’s development. While the Kinect sensor is very powerful, it unfortunately is not very common. On the next page is an image from the March 2011 MaximumPC magazine that shows what is inside of the Kinect. It contains much more than the standard household webcam, or even two webcams. Instead it contains four microphones, an infrared transmitter and camera used to bounce infrared light and perceive depth, a full color camera for facial recognition and video capture, its own processor and more. For obvious reasons this system far exceeds the goal of our project: to use nothing more than a standard webcam (Autopsy).

■ ■ ■ AUTOPSY

Microsoft Kinect

From a technological standpoint, Microsoft's much-ballyhooed Kinect is a modern marvel. The decision to eschew a physical controller in favor of an interface that uses faces, motion, and voice is way ahead of its time, but how does it work? We cracked one open to find out.

COOLING FAN This 3cm fan activates when internal temperatures reach 70 C.

CPU Marvell's AP1-BJDS has been the center of some controversy because no one knows what it is or what it does. Microsoft declined to comment, but did tell us that the Kinect sensor contains a proc each for optical, audio, and motor control.

COLOR CMOS CAMERA This full-color 640x480 camera handles face recognition and provides video-capture functions.

DEPTH PERCEPTION CAMERA The Kinect is able to perceive depth of motion by emitting invisible infrared light from an IR transmitter (left) and then capturing depth in the IR spectrum with a monochrome 640x480 resolution camera (right).

MICROPHONES Three of the Kinect's four-microphone array are located here.

MOTOR AND GEARS This rudimentary set of motor and gears allows the Kinect to tilt on the vertical axis as instructed.

SUBMIT YOUR IDEA Ever wonder what the inside of a power supply looks like? Don't take a chance on destroying your own rig; instead, let us do the dirty work. Tell us what we should crack open for a future autopsy by writing to comments@maximumpc.com.

Lastly, the PlayStation Move has the most in common with the goal and methodology of this project. The Move uses the previously released PlayStation Eye Toy which is truly nothing more than a color webcam with some additional motion sensing technology. The Move uses the Eye Toy in conjunction with remotes to complete the experience. The remotes have a sphere attached to the top of them that light up a different color for each player. The Eye Toy then searches for the colors and once found, observes the size of the sphere to establish a sense of distance. Similar to the Wii remotes, each Move remote contains a three axis accelerometer and a 3 axis angular rate sensor. In addition Move remotes also contain an internal magnetometer that uses earth's magnetic field to provide additional accuracy. The Move was also not available for consumers when this project began, but does end up resembling the final project closely (Anton).

Image Formats

Because the task of capturing an image was already completed, the question of what format to store the image in was the next task. The choice is not limited by common factors such as image quality, color or size because the only use of the images will be a quick comparison to the previous image to decipher what changed or to locate an object. There are many image formats that are quite common and each of them has their own advantages and disadvantages under certain situations. A few of the formats that were considered for use are listed below:

Graphics Interchange Format (GIF) supports a color range between 2 and 256 colors (8 bit) per pixel which is not able to store true color images. GIFs are most commonly used in websites and were designed for that purpose. The advantage of GIF is its indexed image file format which indexes all of the colors visible in the picture. For example, if a picture only contains 16 different colors, each color is represented by a 4-bit pixel instead of the 8 bits required for images containing every color. This can cut the size of an image easily in half without lowering image quality. Additionally GIF also uses a lossless compression to reduce the size of the image even further. Any form of compression such as that used by GIF will complicate any search or comparison algorithms used in the project (Burger).

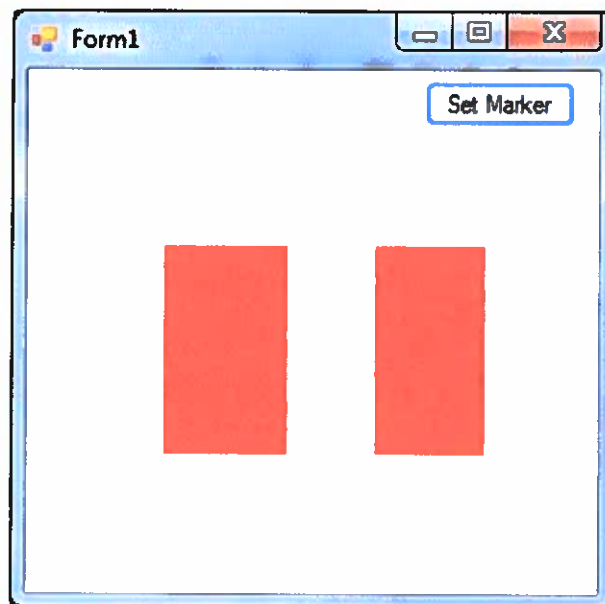
The most common and most widely used format is JPEG, which was named after the group that designed it, the Joint Photographic Experts Group. JPEG is able to display pictures in a continuous format such as the pictures taken by a camera instead of a cartoon image which GIF does well at. JPEG also is able to reduce the file size of an image by a scale of up to 1:25 by essentially blurring and compressing the image in a three step process: Color conversion and down sampling, Cosine transformation and quantization in frequency space, and lossless compression. As long as the image is still detailed enough to identify via software, the loss of quality or blurring is not an issue, but once again any compression will need to be decompressed on the fly to be able to quickly perform comparisons. Because images will be used almost immediately after being captured, the compression is counter-productive (Burger).

While there are many other image formats, almost all of them contain some sort of compression, so the Windows Bitmap format (BMP) became a viable option. A BMP is nothing more than a two dimensional array where each pixel is an entire byte representing what color that pixel should be. This introduces another problem, because a BMP is extremely large and moving large files in real time may be difficult. This is the file format that I decided to implement due to ease of use and in hopes that if the large file size proved to be too much of a problem; I would be easily able to switch to another format (Burger).

Image Comparisons

Because it is very difficult to identify what is on a picture though a computer, Dr. Lin had proposed that we use an image comparison technique. Where we don't necessarily need to know what is moving, but instead are satisfied by knowing that something is moving. This technique will obviously reduce the functionality of the software because you no longer are able to designate multiple items that each control different functions, but it is simple enough to consider.

The ability to detect movement is very simple; it consists of noting more than noting that there was a change between the previously captured image and the newly received one. This would require some sort of imbedded for loop to parse both of the pictures, but the complications arise when a change is detected. Let's presume that we are able to detect that a pixel has changed color from one picture onto the next, that information alone is not enough to decide what direction something has moved, because we do not know what the pixel represents. Consider a large cluster of pixels such as the image displayed below. The red area represents pixels that are found to have changed, while the white area has not noticed any difference from the last frame.



Unfortunately this also doesn't tell us what direction the item in question has moved. Because every time something moves, the place that the item has left will change in addition to the new location of the object, more information is needed. Also let's presume that an object also has a large surface that is entirely the same color. This would result in a section reported as no change even though something moved in it. The image provided on page 7 would be an object that moved either left or right, but has a center that was replaced with the exact same pixel like previously discussed. Because this research was in progress when an alternative method was discovered, no solution for this problem was found.

Algorithm Issues

Throughout early development many other issues were encountered with multiple different algorithms or pieces of hardware. Some of the more notable issues are discussed below:

In addition to the complications about movement detections discussed above, there is the problem of detail. Even the slightest change in lighting or angle can cause an object to change color so drastically that it no longer matches the previously captured color for that item. This actually occurs more than one might initially believe. Presume the user has a standard desk lamp on his/her workspace, this simple lamp will provide a brighter color on one side of the webcam's view than the other. When the user moves an object from one side of the workspace to the other, the item being tracked could be completely lost by a simple algorithm that uses color in too precise of a manner. This issue is not only caused by lamps, but shadows and even a cloud passing outside a window is easily enough to change a pixel's color to another of the 256 colors. For this reason an algorithm that utilizes colors needs to have some sort of leniency and tolerance for these slight changes. In an initial attempt, it was found that even an unnoticeable difference to the human eye, was enough for the computer to feel that items that did not move, suddenly changed.

Web cams themselves also provide a multitude of issues. Because the aim of the software is to be useable by anybody with a webcam, we cannot assume that everybody has the exact same hardware. Even some of the most expensive webcams will provide a somewhat choppy image with things like flicker or lighting changes. This is because the primary use for webcams is communication and the user wants to be seen as clearly as possible. So either through hardware or software a webcam might suddenly change the brightness of images it captures, or decide that it needs to re-focus the image causing new items to become blurry and others to appear crisper without any prompting by the user.

The process of capturing the image also provided an issue. The simplest way to capture a picture from the webcam and store it is to use the Windows Clipboard. The Clipboard is where things are temporarily stored when a user does a function like copy and paste. Unfortunately, the Clipboard is extremely slow and with nothing more than simple captures and display

functionality, a noticeable delay can be observed. Because any movement needs to be noticed quickly and reacted upon accordingly, delay is one of the project's biggest enemies. We would like to aim for the project to work as close to a 30 frames per second as possible. Thirty frames per second is fast enough for the average human to not detect latency, but by using the Clipboard as our primary way of capturing images, this goal is already unobtainable.

In discussions of this project, I often find that people reference the technology of facial recognition and how computers are able to seamlessly identify a face, but there are many differences. For example, facial recognition often requires only one or two images to capture someone's face or its location, but a constant stream of images is required to handle tracking like in this project. Another difference is that most faces look similar, that is they each have two eyes a nose and a mouth, regardless of their shape or size software can predict with relative accuracy where one will appear relative to another. Facial recognition also takes advantage of the fact that eyeballs are almost always the same size and can be easily detected because they are surrounded by small slivers of white on either side of a color with a dark dot in the center. A person's hand, however, will differ greatly because of factors like posture and angle (Face).

TOUCHLESS SDK

Touchless SDK

After the summer of 2010 and the time researching and learning the basics of image processing I stumbled across the Software Development Kit Touchless. Touchless was made by Michael Wasserman who is now a Microsoft employee. He created the project as a college project at Columbia University. The goal of the Touchless project is “to offer users a new and cheap way of experiencing multi-touch capabilities, without the need of expensive hardware or software. All the user needs is a camera, which will track colored markers defined by the user.” Touchless is now a free SDK covered under the Microsoft Public License (Ms-PL) and can be obtained on CodePlex. The goal of the Touchless project is very similar to the goal of this project, and luckily because it is open source, the option of utilizing and modifying it to fit our purposes is available.

Attempts to Alter the SDK

The source code for the SDK is well commented, but takes advantage of some advanced features like multithreading. Initially the idea of modifying Touchless from its current functionality of tracking a user defined color into tracking a hand was the goal. Eventually by studying and learning the source code of the SDK, it became increasingly evident that this might not be possible. The algorithm for searching for a Touchless Marker is not only multithreaded, but is three pages worth of printed out code (the Marker searching code can be found Appendix C) utilizing complex and hard to follow data structures, such as a three dimensional array of hashes, and this is with the simplicity of searching for only a color instead of a shape of pixels that could contain a multitude of colors. A hand tracking algorithm could increase the complexity of the searching algorithm to the point where most of the system’s resources are spent tracking and are no longer available to perform the task that the user wishes to perform through the gestures.

Advantages of Using the SDK

Touchless takes care of many of the previously discussed complications, such as interfacing with the webcam quickly, and storing pictures in an efficient manner. Instead of storing the picture captured by the webcam, Touchless stores only the relevant information such as the X and Y coordinates of the Marker it is trying to track. The SDK also uses C# as its primary language which is extremely useful for accessing system information such as mouse functionality. Another nice feature is that instead of searching for a specific pixel pattern, Touchless is able to track a cluster of pixels and it searches for an average color amongst those pixels. This means that users are able to have markers of any size and can no longer need to worry about complications such as a desk lamp. Lastly and perhaps the best advantage of the SDK is that it is free to use, download and modify under the Ms-PL.

Disadvantages of using the SDK

Simplifying the project to use primarily color tracking was only a minor sacrifice, because Touchless enables Markers to be made of anything distinct enough to track. A user can simply

hold up something like a red pen and wave it about to control the PC instead of a bare hand. Unfortunately, a slight sacrifice of functionality had to be taken to enable the seamless webcam interfacing, and that is that the Touchless SDK only works on Windows based systems that have Direct X installed. So Macintosh or Linux users will not be able to enjoy the gesture controls. Touchless also requires that the programmer use Microsoft Visual Studio which is not exactly an end user sacrifice, but it hinders some of the options while designing the software. Touchless also will only run in a 32 bit format, but because all modern processors are backwards compatible to 32 bit, this is just a speed issue.

Using Touchless

Touchless uses the C# programming language and requires Microsoft Visual Studio. It is very easy to use, but can be difficult to get started. As with any program like this, it is primarily event driven, which is a large change from what most of CSU's classes utilize. The Touchless documentation has a small section about how to add Touchless to your project, and it says:

To add Touchless to an existing visual Studio project, simply right click "References" and select "Add Reference..." go to the browse tab, and select "TouchlessLib.dll". Ensure that both "TouchlessLib.dll" and "WebcamLib.dll" are copied to the same output directory to be used with your builds (Forbes).

I found that this description was not enough to help me get started using Touchless. Particularly with the unfamiliarity of C# and event driven programming, so I created a small project that I would later develop into the final demonstration. I made this program simple enough so I could use it to demonstrate to classmates. It does nothing more than draw dots at the pixel location of a single marker. The process of creating this project is written in the 41 step process included in Appendix B which is a document that I used as a reference while demonstrating Touchless to my classmates.

GESTURE ALGORITHMS

Gesture Algorithms

The discovery of Touchless alleviated a very large portion of work required for this project, but it did not by any means complete the goal. Touchless is able to track something and provide X and Y coordinates representing that object's location, yet doing something with this information and recognizing a gesture out of such is not a trivial task.

Unique gesture problems

Some of the more interesting problems associated with this project come from physical differences between other control devices like a mouse and tracking something from a webcam. One such problem is the dilemma of what to do when you reach the end of your workspace. On a mouse it's as simple as picking up the mouse, moving it to the other end of the workspace (mouse pad) and continuing as normal. For a tracked object like a Touchless marker, you can't really pick it up and reset it. After experimenting with various other options, the only solution that felt somewhat natural, was that of having a second marker indicates whether the first marker's movement should be acted upon or not. This is still not an ideal situation as what would be nice is to have the marker just disappear from view on the webcam and re-appear in a new location. The method I used to solve the issue required a second marker to establish a functionality that mouse users rarely need to think about.

Another significant problem that I initially didn't think of was the problem of how to perform a mouse click. Ideally the rapid movement of the pointer finger would perform this action, but many new complications arose during implementation. One such complication is that people generally are unable to keep the rest of their hand still enough while moving one finger fluently enough to perform a click. This means that every time I tried to click, the cursor would move off of what I'm trying to click on. So to click, a user might need to use a second hand with an additional marker on it for each and every mouse button in addition to the one that was previously mentioned as a sort of "pick up mouse" marker.

When conducting some additional research on this problem, it was found that Microsoft had some similar issues with gesture algorithms. A quote from the Xbox Engineering Blog reads:

Now let's also consider the complexity of recognizing gestures with a seemingly simple movement like a push. When you initially think of pushing your hand out to select, you likely think of the direction of the push being completely perpendicular to the plane of your body. In reality, you don't push away from your body in a straight line because of how your shoulder and arm joints are designed. The result of this nuance is that every human performs a push slightly differently, but everyone expects that their action will result in the same outcome – a push. Successful gesture recognition is about understanding the nuances in the way humans perform interactions and teaching technology to be mindful of these differences.

Microsoft overcame this issue by testing a large amount of people to test a variety of different algorithms and report which movements felt the most natural. Unfortunately, without having a large population to test on, or the time to invest in this sort of research, this is not an option.

A comparative algorithm

Two primary algorithms were implemented to various extents to move the cursor on a computer screen. First is the comparative algorithm which reacted according to the distance that a marker moved in comparison to the last time the marker was seen. A simple version of that code is displayed below:

```
void m_OnChange(object sender, MarkerEventArgs e)
{
    if ((e.EventData.X > e.EventMarker.LastGoodData.X + 3) || (e.EventData.X <
e.EventMarker.LastGoodData.X - 3))//marker went right
    {
        Cursor.Position = new Point(Cursor.Position.X +
(e.EventMarker.LastGoodData.X - e.EventData.X)*3 , Cursor.Position.Y);
    }
    if ((e.EventData.Y > e.EventMarker.LastGoodData.Y + 3) ||
(e.EventData.Y < e.EventMarker.LastGoodData.Y - 3))//marker went down
    {
        Cursor.Position = new Point(Cursor.Position.X, Cursor.Position.Y
+ (e.EventData.Y - e.EventMarker.LastGoodData.Y) * 3);
    }
}
```

Due to camera shake and marker instability the marker was set to require a change of at least three pixels, this is why the if statement includes a + or - 3. The cursor would then be set according to the distance changed with a multiplier to make the change worth a significant number of pixels. Unfortunately, Marker reliability became an issue. A Marker would occasionally not be found and cause the previous data to become skewed or a change to be measured that either wasn't there or is much larger than it should be.

A relative algorithm

Another approach that is even more simplistic involves only the use of the currently observed coordinates. Instead of comparing the position to its last known location, we simply place the cursor in a location relative to its location on within the webcam's field of view. Obtaining that location done through fairly simple scaling with the resolution of the user's screen obtained through the system. This was done with one single line of code, although it is somewhat lengthy.

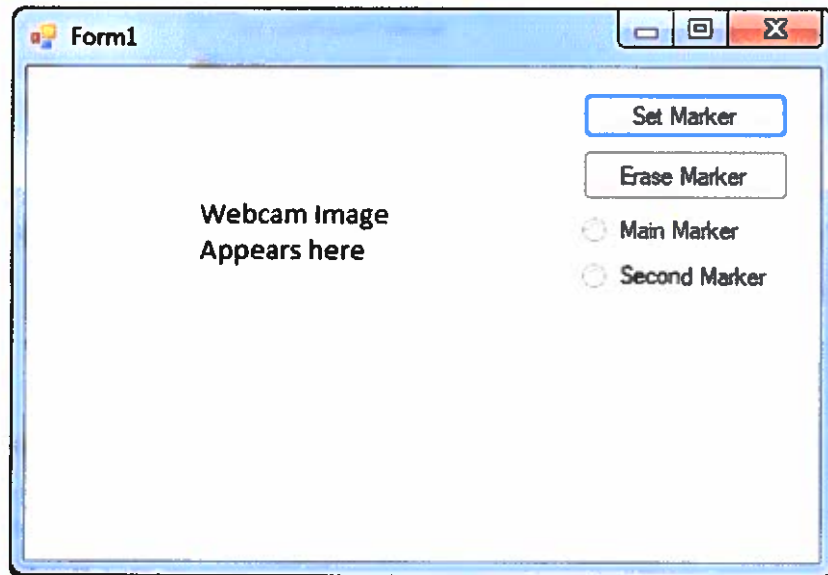
```

void m_OnChange(object sender, MarkerEventArgs e)
{
    if (e.EventData.Present)
    {
        Cursor.Position = new
        Point(System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width -
        e.EventData.X *
        (System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width /
        pictureBox1.Width), e.EventData.Y *
        (System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height /
        pictureBox1.Height));
    }
}

```

The division and multiplications make this algorithm efficient regardless of screen resolution or the size of the image being utilized by the webcam. The equation also requires a subtraction because of a mirror-like effect caused by the difference between the user's view and the webcam's view, otherwise when the user moves a Marker to the left the cursor would actually go to the right.

POTENTIAL IMPROVEMENTS



Additional features

At the submission of the project, only a curser's and the left click function are implemented. Further implementations would be simple to program, but would require additional gestures and/or Markers. It might be hard for the average user to find more than a handful of brightly colored and easily tracked items in their household, so these features could be somewhat challenging to implement.

More Fun Ideas

The scope of this project can easily be expanded to incorporate many different ideas such as games, or more specific software like PowerPoint presentations. During my rsearch I found that many people are using Wii Remotes and their Bluetooth infrared tracking technology to create interesting programs. The only reason for the Wii Remotes in these projects is to simplify the tracking process to infrared which is all that the Wii Remote can sense. But because the Touchless SDK is able to track almost any color using nothing more than a standard webcam, we could potentially port these project over and remove the necessity for the Wii Remote.

One particular project by Johnny Chung Lee uses the Wii Remote for tracking of a user's head and is able to create a very stunning three dimensional affect. Without having done much research on the project, I'm unsure if this project would be accurate enough, but it would be very interesting to research. Johnny Chung Lee's project can be found on his website at <http://johnnylee.net/projects/wii/>.

APPENDIX A

```
Public Class Form1
```

```
    Const WM_CAP As Short = &H4005
    Const WM_CAP_DRIVER_CONNECT As Integer = WM_CAP + 10
    Const WM_CAP_DRIVER_DISCONNECT As Integer = WM_CAP + 11
    Const WM_CAP_EDIT_COPY As Integer = WM_CAP + 30
    Const WM_CAP_SET_PREVIEW As Integer = WM_CAP + 50
    Const WM_CAP_SET_PREVIEWRATE As Integer = WM_CAP + 52
    Const WM_CAP_SET_SCALE As Integer = WM_CAP + 53
    Const WS_CHILD As Integer = &H40000000
    Const WS_VISIBLE As Integer = &H10000000
    Const SWP_NOMOVE As Short = &H2S
    Const SWP_NOSIZE As Short = 1
    Const SWP_NOZORDER As Short = &H4S
    Const HWND_BOTTOM As Short = 1
```

```
    Dim iDevice As Integer = 0 ' Normal device ID
    Dim hWndd As Integer ' Handle value to preview window
```

```
    ' Declare function from AVI capture DLL.
```

```
    Declare Function SendMessage Lib "user32" Alias "SendMessageA" _
        (ByVal hWndd As Integer, ByVal wParam As Integer, ByVal lParam As Integer, _
        ByVal lParam As Object) As Integer
```

```
    Declare Function SetWindowPos Lib "user32" Alias "SetWindowPos" (ByVal hWndd As Integer, _
        ByVal hWndInsertAfter As Integer, ByVal x As Integer, ByVal y As Integer, _
        ByVal cx As Integer, ByVal cy As Integer, ByVal wFlags As Integer) As Integer
```

```
    Declare Function DestroyWindow Lib "user32" (ByVal hWndd As Integer) As Boolean
```

```
    Declare Function capCreateCaptureWindowA Lib "avicap32.dll" _
        (ByVal lpszWindowName As String, ByVal dwStyle As Integer, _
        ByVal x As Integer, ByVal y As Integer, ByVal nWidth As Integer, _
        ByVal nHeight As Short, ByVal hWndParent As Integer, _
        ByVal nID As Integer) As Integer
```

```
    Declare Function capGetDriverDescriptionA Lib "avicap32.dll" (ByVal wDriver As Short, _
        ByVal lpszName As String, ByVal cbName As Integer, ByVal lpszVer As String, _
        ByVal cbVer As Integer) As Boolean
```

```
    ' Connect to the device.
```

```
    Private Sub LoadDeviceList()
```

```
        Dim strName As String = Space(100)
        Dim strVer As String = Space(100)
        Dim bReturn As Boolean
        Dim x As Integer = 0
```

```
        ' Load name of all available devices into the lstDevices .
```

```
        Do
```

```
            ' Get Driver name and version
            bReturn = capGetDriverDescriptionA(x, strName, 100, strVer, 100)
            ' If there was a device add device name to the list
            If bReturn Then
                lstDevices.Items.Add(strName.Trim)
```

```
            End If
```

```
            x += 1
```

```
        Loop Until bReturn = False
```

```
    End Sub
```

```
    ' To display the output from a video capture device, you need to create a capture window.
```

```
    Private Sub OpenPreviewWindow()
```

```
        Dim iHeight As Integer = picCapture.Height
```

```

Dim iWidth As Integer = picCapture.Width

' Open Preview window in picturebox .
' Create a child window with capCreateCaptureWindowA so you can display it in a picturebox.

hHwnd = capCreateCaptureWindowA(iDevice, WS_VISIBLE Or WS_CHILD, 0, 0, 640, _
    480, picCapture.Handle.ToInt32, 0)

' Connect to device
If SendMessage(hHwnd, WM_CAP_DRIVER_CONNECT, iDevice, 0) Then

    ' Set the preview scale
    SendMessage(hHwnd, WM_CAP_SET_SCALE, True, 0)

    ' Set the preview rate in milliseconds
    SendMessage(hHwnd, WM_CAP_SET_PREVIEWRATE, 66, 0)

    ' Start previewing the image from the camera
    SendMessage(hHwnd, WM_CAP_SET_PREVIEW, True, 0)

    ' Resize window to fit in picturebox
    SetWindowPos(hHwnd, HWND_BOTTOM, 0, 0, picCapture.Width, picCapture.Height, _
        SWP_NOMOVE Or SWP_NOZORDER)

    btnStop.Enabled = True
    btnStart.Enabled = False
Else
    ' Error connecting to device close window
    DestroyWindow(hHwnd)
    btnStart.Enabled = False
End If
End Sub

' Use SendMessage to copy the data to the clipboard Then transfer the image to the picture box.
Private Sub btnSave_Click(ByVal sender As System.Object, ByVal e As System.EventArgs)
    Dim data As IDataObject
    Dim bmap As Image
    ' Copy image to clipboard
    SendMessage(hHwnd, WM_CAP_EDIT_COPY, 0, 0)

    ' Get image from clipboard and convert it to a bitmap
    data = Clipboard.GetDataObject()
    If data.GetDataPresent(GetType(System.Drawing.Bitmap)) Then
        bmap = CType(data.GetData(GetType(System.Drawing.Bitmap)), Image)
        PicCapture.Image = bmap
        ClosePreviewWindow()
        btnStart.Enabled = False
        btnStop.Enabled = False
        btnStart.Enabled = True
        'If sfdImage.ShowDialog = DialogResult.OK Then
        '    bmap.Save(sfdImage.FileName, Imaging.ImageFormat.Bmp)
        'End If
    End If
End Sub

' Finally, to close the preview window, disconnect from the device and destroy the preview window.
Private Sub ClosePreviewWindow()
    ' Disconnect from device
    SendMessage(hHwnd, WM_CAP_DRIVER_DISCONNECT, iDevice, 0)

    ' close window
    DestroyWindow(hHwnd)
End Sub

Private Sub btnStart_Click(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles btnStart

```

```
.Click
    lstDevices.Items.Clear()
    LoadDeviceList()
    OpenPreviewWindow()
End Sub

Private Sub btnStop_Click(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles btnStop. ✓
Click
    ClosePreviewWindow()
    btnStart.Enabled = True
    btnStop.Enabled = False
End Sub

Private Sub Form1_FormClosing(ByVal sender As System.Object, ByVal e As System.Windows.Forms. ✓
FormClosingEventArgs) Handles MyBase.FormClosing
    ClosePreviewWindow()
End Sub
End Class
```

APPENDIX B

1. Create a C# Windows Form Application
2. Add the touchlesslib.dll as a reference by right clicking references and selecting add reference.
3. Add using TouchlessLib
4. Add using System.Drawing.Imaging

```
using TouchlessLib; //the touchless library
using System.Drawing.Imaging; //for bitmap functionality
```

5. Create a load event handler by simply double clicking anywhere on the blank Form
6. Make an instance of Touchless within the partial class:

```
TouchlessMgr _touch = new TouchlessMgr(); //create a touchless instance
```

7. In the load handler select the first != null camera using a foreach loop
8. Set _touch.CurrentCamera = c
9. Create a c.OnImageCaptured event by typing c.OnImageCaptured +=... and using tab a lot

```
private void Form1_Load(object sender, EventArgs e)
{
    foreach (Camera c in _touch.Cameras) //for every camera that touchless found
    {
        if (c != null) //check if its working
        {
            _touch.CurrentCamera = c; //set the current camera to the camera we found
            c.OnImageCaptured += new EventHandler<CameraEventArgs>(c_OnImageCaptured); //an event
            _overlay = new Bitmap(c.CaptureWidth, c.CaptureHeight, PixelFormat.Format24bppRgb);
            _overlay.MakeTransparent(); //makes the overlay transparent very easily!
            break; //break because we are only using one camera
        }
    }
    pictureBox1.Paint += new PaintEventHandler(pictureBox1_Paint); //created the paint event
}
```

10. Remove the default implementation of the handler that visual put in.
11. Create a variable bitmap _b for saving the image from the camera. Type Bitmap _b; dur
12. Create a picture box on the form for us to use to display stuff on
13. Change the picture box's Background ImageLayout from tile to None.
14. In the onImageCapture even set _b to e.image.
15. Create a pictureBox1.paint += event in the load event) as the image above
16. In the onImageCapture event make pictureBox1.invalidate();
17. In the paint method make sure _b != null; see below
18. Draw the image _b with the code below.

```

void pictureBox1_Paint(object sender, PaintEventArgs e)
{
    lock (this)
    {
        if (_b != null) //if the image _b has been captured
        {
            e.Graphics.DrawImageUnscaledAndClipped(_b, pictureBox1.ClientRectangle); //draw
            e.Graphics.DrawImageUnscaledAndClipped(_overlay, pictureBox1.ClientRectangle);
        }
    }
}

```

19. Change the project to an x86 project by right clicking on the project in solution explorer and going to properties/bold/platformTarget
20. Built the project
21. Put the webcam.lib into the project's bin/depug directory
22. Run to test the webcam is working
23. Create an add marker button on the form
24. Create a button1_click event by double clicking on the button in form view
25. In there make another bitmap C = _touch.currentCamera.getImage

```

private void button1_Click(object sender, EventArgs e)
{
    Bitmap c = _touch.CurrentCamera.GetCurrentImage(); //:
    pictureBox1.BackgroundImage = c; //set that image to i
    button1.Enabled = false;
}

```

26. Make pictureBox1.BackgroundImage = the new image c
27. Disable the button
28. In the onImageCaptured handler, enclose all of it in an if that checks if button1.enabled

```

void c_OnImageCaptured(object sender, CameraEventArgs e)
{
    if (button1.Enabled) //make sure the user isn't trying
    {
        _b = e.Image; //save the image captured to _b
        pictureBox1.Invalidate(); //this causes the pictureb
    }
}

```

29. Create a mouseClicked event in the pictureBox
30. Within the mouse click event create a maerk m = _touch.addmaerker(...see below)
31. Set m.highlight=true so we can see it
32. Create a m.onChange even handler the same way we did before
33. Re-enable button1

```

private void pictureBox1_MouseClick(object sender, MouseEventArgs e)
{
    Marker m = _touch.AddMarker("marker", (Bitmap)pictureBox1.BackgroundImage, new Point(e.X, e.Y), 5);
    //created a new marker M using the background image that we saved and froze (typecasted to bitmap)
    //and made it a hardcoded 5 pixels in size.
    m.Highlight = true; //this surrounds the maker with a box when we run
    m.OnChange += new EventHandler<MarkerEventArgs>(m_OnChange); // create a new event for when the #
    button1.Enabled = true; //re enable the button because we are now done making a marker
}

```

34. Create another bitmap named _overlay
35. In the if inside of foreach _overlay = new Bitmap(c.CaptureWidth, c.CaptureHeight, PixelFormat.Format24bppRgb);
36. Make the overlay transparent with the .makeTransparent() method
37. Add the drawing of _overlay to the pictureBox1 paint event
38. In the m_onChange event set a pickle to Black with _overlay.SetPixel(e.EventData.X, e.EventData.Y, Color.Black);
39. Enclose the previous statement in an if(e.eventData.Present)
40. Add a lock statement to ensure we don't try to print the picture box at the same time
41. Add a lock to the pain method lock(this)

```

void m_OnChange(object sender, MarkerEventArgs e)
{
    lock (this) //make sure we don't paint at the same time we are changing
    {
        //this is where we can do stuff every time the marker changes!!
        if (e.EventData.Present) //make sure the marker is currently being s
        {
            _overlay.SetPixel(e.EventData.X, e.EventData.Y, Color.Black); //
        }
    }
}

```

```

using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Windows.Forms;
using TouchlessLib;//the touchless library
using System.Drawing.Imaging;//for bitmap functionality

namespace SampleFromTutorial
{
    public partial class Form1 : Form
    {
        public Form1()
        {
            InitializeComponent();

            TouchlessMgr _touch = new TouchlessMgr();//create a touchless instance
            Bitmap _overlay;//this is for the pixels we change later

            private void Form1_Load(object sender, EventArgs e)
            {
                foreach (Camera c in _touch.Cameras)//for every camera that touchless found
                {
                    if (c != null)//check if its working
                    {
                        _touch.CurrentCamera = c;//set the current camera to the camera we found
                        c.OnImageCaptured += new EventHandler<CameraEventArgs>(c_OnImageCaptured);//an event
handler to fire every time the new webcam gets an image ✓
                        _overlay = new Bitmap(c.CaptureWidth, c.CaptureHeight, PixelFormat.Format24bppRgb);//
use the bitmap constructor to set width, height, and 24bit RGB pixles ✓
                        _overlay.MakeTransparent();//makes the overlay transparent very easily!
                        break;//break becasue we are only using one camera
                    }
                }
                pictureBox1.Paint += new PaintEventHandler(pictureBox1_Paint);//created the paint event
            }

            void pictureBox1_Paint(object sender, PaintEventArgs e)
            {
                lock (this)//
                {
                    if (_b != null)//if the image _b has been captured
                    {
                        e.Graphics.DrawImageUnscaledAndClipped(_b, pictureBox1.ClientRectangle);//draw it on
the picture box ✓
                        e.Graphics.DrawImageUnscaledAndClipped(_overlay, pictureBox1.ClientRectangle);//draw
the overlay in the picturebox ✓
                    }
                }
            }

            Bitmap _b;//a variable to store the new image we are going to get from the camera
            void c_OnImageCaptured(object sender, CameraEventArgs e)
            {
                if (button1.Enabled)//make sure the user isn't trying to set a marker
                {
                    _b = e.Image;//save the image captured to _b
                    pictureBox1.Invalidate();//this causes the picturebox to re paint (calling the paint event)
                }
            }
        }
    }
}

```

```

private void button1_Click(object sender, EventArgs e)
{
    Bitmap c = _touch.CurrentCamera.GetCurrentImage();//save the image currently on the camera
    pictureBox1.BackgroundImage = c;//set that image to the background of pictureBox1
    button1.Enabled = false;
}

private void pictureBox1_MouseClick(object sender, MouseEventArgs e)
{
    Marker m = _touch.AddMarker("marker",(Bitmap)pictureBox1.BackgroundImage,new Point(e.X, e.Y),5) ✓
;
    //created a new marker M using the background image that we saved and froze (typecasted to ✓
    bitmap), give it the point where the mouse is
    //and made it a hardcoded 5 pixels in size.
    m.Highlight = true;//this surrounds the maker with a box when we run
    m.OnChange += new EventHandler<MarkerEventArgs>(m_OnChange); // create a new event for when the ✓
    marker changes
    button1.Enabled = true; //re enable the button because we are now done makeing a marker
}

void m_OnChange(object sender, MarkerEventArgs e)
{
    lock (this)//make sure we don't paint at the same time we are changeing
    {
        //this is where we can do stuff every time the marker changes!!
        if (e.EventData.Present)//make sure the marker is currently being seen in the image
        {
            _overlay.SetPixel(e.EventData.X, e.EventData.Y, Color.Black);//change the pixel at the ✓
            new marker location to black
        }
    }
}
}
}
}
}

```

```

using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Windows.Forms;
using TouchlessLib;//the touchless library
using System.Drawing.Imaging;//for bitmap functionality

namespace SampleFromTutorial
{
    public partial class Form1 : Form
    {
        public Form1()
        {
            InitializeComponent();

            TouchlessMgr _touch = new TouchlessMgr();//create a touchless instance
            Bitmap _overlay;//this is for the pixels we change later

            //*****Mouse Click Stuff*****
            [System.Runtime.InteropServices.DllImport("user32.dll")]
            public static extern void mouse_event(int dwFlags, int dx, int dy, int cButtons, int dwExtraInfo);

            public const int MOUSEEVENTF_LEFTDOWN = 0x02;
            public const int MOUSEEVENTF_LEFTUP = 0x04;
            public const int MOUSEEVENTF_RIGHTDOWN = 0x08;
            public const int MOUSEEVENTF_RIGHTUP = 0x10;
            //*****End Mouse Click Stuff*****

            private void MouseClick()
            {
                int x = 100;
                int y = 100;

                mouse_event(MOUSEEVENTF_LEFTDOWN, x, y, 0, 0);
                mouse_event(MOUSEEVENTF_LEFTUP, x, y, 0, 0);
            }

            private void Form1_Load(object sender, EventArgs e)
            {
                foreach (Camera c in _touch.Cameras)//for every camera that touchless found
                {
                    if (c != null)//check if its working
                    {
                        _touch.CurrentCamera = c;//set the current camera to the camera we found
                        c.OnImageCaptured += new EventHandler<CameraEventArgs>(c_OnImageCaptured);//an event
handler to fire every time the new webcam gets an image
                        _overlay = new Bitmap(c.CaptureWidth, c.CaptureHeight, PixelFormat.Format24bppRgb);//
use the bitmap constructor to set width, height, and 24bit RGB pixels
                        _overlay.MakeTransparent();//makes the overlay transparent very easily!
                        break;//break becasue we are only using one camera
                    }
                }
                pictureBox1.Paint += new PaintEventHandler(pictureBox1_Paint);//created the paint event
            }

            void pictureBox1_Paint(object sender, PaintEventArgs e)
            {
                lock (this)//

```

```

        {
            if (_b != null)//if the image _b has been captured
            {
                e.Graphics.DrawImageUnscaledAndClipped(_b, pictureBox1.ClientRectangle);//draw it on ✓
the picture box
                e.Graphics.DrawImageUnscaledAndClipped(_overlay, pictureBox1.ClientRectangle);//draw ✓
the overlay in the picturebox
            }
        }

        Bitmap _b;//a variable to store the new image we are going to get from the camera
        void c_OnImageCaptured(object sender, CameraEventArgs e)
        {
            if (button1.Enabled)//make sure the user isn't trying to set a marker
            {
                _b = e.Image;//save the image captured to _b
                pictureBox1.Invalidate();//this causes the picturebox to re paint (calling the paint event)
            }
        }

        private void button1_Click(object sender, EventArgs e)
        {
            Bitmap c = _touch.CurrentCamera.GetCurrentImage();//save the image currently on the camera
            pictureBox1.BackgroundImage = c;//set that image to the background of pictureBox1
            button1.Enabled = false;
        }

        private void pictureBox1_MouseClick(object sender, MouseEventArgs e)
        {
            if (radioButton1.Checked)
            {
                Marker m = _touch.AddMarker("marker", (Bitmap)pictureBox1.BackgroundImage, new Point(e.X, e.Y), 5);
                //created a new marker M using the background image that we saved and froze (typecasted to ✓
                bitmap), give it the point where the mouse is
                //and made it a hardcoded 5 pixels in size.
                m.Highlight = true;//this surrounds the maker with a box when we run
                m.OnChange += new EventHandler<MarkerEventArgs>(m_OnChange); // create a new event for when ✓
the marker changes
                button1.Enabled = true; //re enable the button because we are now done makeing a marker
            }
            else if (radioButton2.Checked)
            {
                Marker n = _touch.AddMarker("nmarker", (Bitmap)pictureBox1.BackgroundImage, new Point(e.X, ✓
                e.Y), 5);
                //created a new marker M using the background image that we saved and froze (typecasted to ✓
                bitmap), give it the point where the mouse is
                //and made it a hardcoded 5 pixels in size.
                n.Highlight = true;//this surrounds the maker with a box when we run
                n.OnChange += new EventHandler<MarkerEventArgs>(n_OnChange); // create a new event for when ✓
the marker changes
                button1.Enabled = true; //re enable the button because we are now done makeing a marker
            }
        }

        void n_OnChange(object sender, MarkerEventArgs e)
        {
            if (e.EventData.Present && !e.EventMarker.PreviousData.Present)//notes when the marker just ✓
            became visable
            {
                MouseClick();
            }
        }

        void m_OnChange(object sender, MarkerEventArgs e)

```

```

    {
        // lock (this)//make sure we don't paint at the same time we are changing
        //{
            //this is where we can do stuff every time the marker changes!!
            if (e.EventData.Present)//make sure the marker is currently being seen in the image
            {
                //_overlay.SetPixel(e.EventData.X, e.EventData.Y, Color.Black);//change the pixel at
the new marker location to black ✓

                Cursor.Position = new Point(System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width - e. ✓
EventData.X * (System.Windows.Forms.Screen.PrimaryScreen.Bounds.Width / pictureBox1.Width), e.EventData ✓
.Y * (System.Windows.Forms.Screen.PrimaryScreen.Bounds.Height / pictureBox1.Height));

                //if ((e.EventData.X > e.EventMarker.PreviousData.X + 3) || (e.EventData.X < e. ✓
EventMarker.PreviousData.X - 5))//maker went right
                //{
                    //    Cursor.Position = new Point(Cursor.Position.X + (e.EventMarker.PreviousData.X - e ✓
.EventData.X)*3 , Cursor.Position.Y);
                    //}
                //if ((e.EventData.Y > e.EventMarker.PreviousData.Y + 3) || (e.EventData.Y < e. ✓
EventMarker.PreviousData.Y - 5))//maker went down
                //{
                    //    Cursor.Position = new Point(Cursor.Position.X, Cursor.Position.Y + (e.EventData.Y ✓
- e.EventMarker.PreviousData.Y)*3);
                    //}

                // }
            }
        }

        private void button2_Click(object sender, EventArgs e)
        {
            for (int i = 0; i < _touch.MarkerCount; i++)
            {
                _touch.RemoveMarker(i);
            }
        }
    }
}

```

APPENDIX C

Touchless SDK Documentation

Homepage: <http://www.codeplex.com/touchless>

Project Author: **Michael Wasserman**

Michael.Wasserman@microsoft.com

Thanks due to: **Gary Caldwell, Isabel Mattos, Nicole Steinbok, Quinn Hawkins, MS Office Labs Community Projects, John Kender, and you!**

0. Table of contents

- 0. Table of contents (oh no! infinite loop!)**
- 1. What's Touchless?**
- 2. What's in this release?**
- 3. What's in the source repository?**
- 4. How to add Touchless to your project**
- 5. Reference Information**
- 6. Backlog of ideas**

1. What's Touchless?

Touchless is an SDK that allows users to create and experience multi-touch applications. Touchless started as Mike Wasserman's college project at Columbia University. The main idea: to offer users a new and cheap way of experiencing multi-touch capabilities, without the need of expensive hardware or software. All the user needs is a camera, which will track colored markers defined by the user.

Mike presented the project at the Microsoft Office Labs Productivity Science Fair, Office Labs fell in love with it, and Touchless was chosen as a Community Project. Our deliverables include an extensible demo application to showcase a limited set of multi-touch capabilities, but mainly we are delivering an SDK to allow users to build their own multi-touch applications.

Now, Touchless is released free and open-source to the world under the Microsoft Public License (Ms-PL) on CodePlex. Our goals are to drive community involvement and use of the SDK as it continues to develop.

Remember that this is just the beginning; and you're invited to join our journey. Send us

your questions and feedback, use Touchless SDK in your .NET applications and XNA games, and support the community by contributing to the source code.

Unleash the power of your webcam!

2. What's in this release?

This release includes a short list of binary files to demonstrate Touchless:

- WebCamLib.dll - Interfaces with DirectShow to grab webcam frames
- TouchlessLib.dll - Contains the functionality of Touchless SDK
- TouchlessDemo.exe - A small demonstration of what Touchless can be used for
- Image.gif - Used in TouchlessDemo's image demo
- Touchless.chm - A documentation file of the Touchless API for developers
- Touchless.jpg - A class diagram of the Touchless API for developers
- Touchless.rtf - This file; you're reading it right now!

If you just want to try out the simple demos offered with the SDK, extract the release to any location on your computer, and run "TouchlessDemo.exe".

3. What's in the source repository?

The source repository is hosted at the homepage of Touchless:

<http://www.codeplex.com/touchless>

The directories you'll find there are:

- bin - Binary versions of the files, updated more frequently than our releases
- Documentation - Browse media documenting the project
- Samples - Apps/Games that demonstrate or use Touchless (submit yours!)
- TouchlessLib - Touchless functionality, written in C#, uses .NET 3.0+, VS2005+

- TouchlessUnitTests - Unit tests to prevent regression
- WebCamLib - Webcam functionality, written in C++, uses DX SDK Aug. 2007

Become a contributor by adding your Touchless apps or improving the SDK code!

4. How to add Touchless to your project

By adding Touchless to your project, you can give your users a truly fun, novel, and functional multi-touch experience, and all they need is a webcam!

The prerequisites you'll need to develop using Touchless are:

- Visual Studio 2005 or 2008
- .NET 3.0 or higher
- "TouchlessLib.dll" and "WebcamLib.dll"

To add Touchless to an existing visual Studio project, simply right click "References" and select "Add Reference..." go to the browse tab, and select "TouchlessLib.dll". Ensure that both "TouchlessLib.dll" and "WebcamLib.dll" are copied to the same output directory to be used with your builds.

5. Reference Information

You can find extensive information on all properties and methods of each class in our XML-generated documentation file "TouchlessLib.chm". The Touchless class diagram reflects public classes, methods, and properties of the API; it is available as "Touchless.jpg". These files are distributed with each release, and also available in the source code repository, located in the documentation folder.

6. Backlog of Ideas

This is a very terse list of ideas for improving Touchless SDK.

- ColorLib
 - Improve HSV colorspace partitioning model. We could group perceived similar colors better. Potentially replace with a group clustering algorithm. Perhaps just

refine the per-dimension bin counts, or replace the hash function.

- Use a lookup table instead of transforming RGB to HSV. We can just terminate early if it's not in the lookup table.
- Reduce loop overhead of converting ARGB values into RGB values, then into HSV values, then into Binned HSV values, then finally into a hash # for color lookup during marker update. Potentially use a lookup table for a subset of colors to avoid the math altogether.
- Improve HSV color grouping, consider refining the per-dimension bin counts or using a different HSV color-space partitioning model that better suits human perception of similar colors.

- **Marker**

- Implement a way of getting higher degree moments of inertia. Mostly, we are interested in the axis of least rotational momentum and the roundness factor.
- Allow the user to send a mask image with the add marker bitmap for arbitrary marker region selections."
- Extend or replace alpha smoothing with exponential decay to provide smoothed marker data and reduce the marker jumpiness.
- Optimize threshold, or replace threshold concept with a partial matching. Also, step threshold by numbers that actually make a difference, or just have sensitivity +/- buttons and increment functions
- Expose smoothing factor as a public marker property.
- Fix and improve the automated marker tests
- Standardize some marker colors, create an "auto-find makers"
- Improve the search bounds of a previously absent marker.
- Improve the meta-tracking (cases where small numbers of pixels are missing from the middle of a marker, or are outliers of the concentration of pixels)
- Periodically/continuously adopt surrounding pixels of confirmed marker pixels
- Coloravg is currently just marker representative color. Implement a way of actually getting a color average from the set of colors found
- Improve Marker highlighting
- Improve upon the raster scan algorithm used for marker updating.
- Optimize the method for getting the marker appearance from a circular area of a bitmap; we could use hierarchical bounds intersection or something smarter than the current scan algorithm.
- Optimize the values used to increment/decrement color frequencies for marker appearance detection. This should be somehow based on signal/noise ratios.
- Improve the expected marker regions used for scanning on update. We could consider the marker's acceleration, rather than just the velocity. Perhaps try using regions that aren't axis-aligned rectangles.

- **TouchlessMgr**

- Add functionality to save and load marker configuration files (reduce repeat

training of the same marker, possibly provide autoconfig files for standard markers... will variant lighting allow for this?)

- Implement additional marker data such as ColorAverage, ColorSpace, Axis and Roundness.
- Add flood fill algorithm so we can add a marker with a few points in the Bitmap.
- Refine the marker tracked colors as we find colors around the marker.
- The representative color doesn't always match the perceived color of the marker.
- Provide subsequent examples of a marker appearance
- Have TouchlessMgr actually expose a way to get a list of the current markers
- Make a better exception for camera start failure
- Validate the PixelFormat of incoming images.
- Create a utility function to retrieve ImageData in a consistent manner; we have a bit of code duplication right now.
- Make a public interface for demo classes to implement, then allow the user to just invoke start and stop of a demo class on the library
- Standardize error handling and exception generation across the project.

Camera Class

Properties

- CaptureHeight
- CaptureWidth
- Fps

Methods

- Dispose
- GetCurrentImage
- ShowPropertiesDialog
- ToString

Events

- OnImageCaptured

TouchlessMgr Class

Properties

- Cameras
- CurrentCamera
- MarkerCount
- Markers

Methods

- AddMarker
- Dispose
- RefreshCameraList
- RemoveMarker
- TouchlessMgr

Marker Class

Properties

- CurrentData
- Highlight
- LastGoodData
- Name
- PreviousData
- ProvideCalculatedProperties
- RepresentativeColor
- SmoothingEnabled
- Threshold

Methods

- ToString

Events

- OnChange

MarkerEventArgs Class

EventArgs

- Properties
 - EventData
 - EventMarker

MarkerEventData Struct

Properties

- Area
- Bounds
- ColorAvg
- DX
- DY
- Present
- Timestamp
- X
- Y

CameraEventArgs Class

EventArgs

- Properties
 - Image

```

internal void UpdateMarkers(Bitmap image)
{
    if (_markers == null || _markers.Count == 0)
        return;

    // Allow use of pointers
    unsafe
    {
        // TODO: Validate image.PixelFormat
        // TODO: push this to a function like Camera.getImageData();
        Rectangle rect = new System.Drawing.Rectangle(0, 0, image.Width, image.Height);
        BitmapData data = image.LockBits(rect, ImageLockMode.ReadWrite, image.PixelFormat);
        int* pPixel0 = (int*)data.Scan0, pPixelCurr;

        HSV hsv = new HSV();
        bool dirtyHSV = true;
        ColorKey key = new ColorKey();

        // Make a copy of _markers for thread safety
        Marker[] markers = new Marker[_markers.Count];
        _markers.CopyTo(0, markers, 0, markers.Length);

        // Prepare the markers for update and populate the y-axis scan command list
        // TODO: smarter updating of command lists between consecutive images?
        _markerScanCommandsY.Clear();
        foreach (Marker marker in _markers)
        {
            preprocessMarker(marker, image.Width, image.Height);
            _markerScanCommandsY.Add(marker.searchMinY);
            _markerScanCommandsY.Add(marker.searchMaxY);
        }

        // Sort the y-axis scan command list
        _markerScanCommandsY.Sort(_MarkerScanCommandComparison);

        // Loop over the y-axis scan command list
        for (int mscIndexY = 0; mscIndexY < _markerScanCommandsY.Count; mscIndexY++)
        {
            // Add or remove the marker for consideration in the x-axis command list
            if (_markerScanCommandsY[mscIndexY].command == ScanCommand.addMarker)
                _scanMarkersY.Add(_markerScanCommandsY[mscIndexY].marker);
            else /*if(_markerScanCommandsY[mscIndexY].scanCommand == ScanCommand.remMarker)*/
                _scanMarkersY.Remove(_markerScanCommandsY[mscIndexY].marker);

            // If we have no markers to scan in this set of rows, jump to the next y-axis scan
command (continue)
            if (_scanMarkersY.Count == 0)
                continue;

            // Populate the x-axis scan command list from the y-axis list of markers for
consideration
            // TODO: smarter updating of command lists between row sections?
            _markerScanCommandsX.Clear();
            foreach (Marker marker in _scanMarkersY)
            {
                _markerScanCommandsX.Add(marker.searchMinX);
                _markerScanCommandsX.Add(marker.searchMaxX);
            }

            // Sort the x-axis scan command list
            _markerScanCommandsX.Sort(_MarkerScanCommandComparison);

            // Loop over the x-axis scan command list
            for (int mscIndexX = 0; mscIndexX < _markerScanCommandsX.Count; mscIndexX++)
            {
                // Add or remove the marker for consideration in the pixel scanning

```

```

        if (_markerScanCommandsX[mscIndexX].command == ScanCommand.addMarker)
            _scanMarkersX.Add(_markerScanCommandsX[mscIndexX].marker);
        else /*if(_markerScanCommandsX[mscIndexX].scanCommand == ScanCommand.remMarker)*/
            _scanMarkersX.Remove(_markerScanCommandsX[mscIndexX].marker);

        // If we have no markers to scan in this area of the row, jump to the next x-axis ✓
scan command (continue)
        if (_scanMarkersX.Count == 0)
            continue;

        // Check for proper looping conditions
        if( _markerScanCommandsY.Count < mscIndexY+1 || _markerScanCommandsX.Count <
mscIndexX+1 )
            throw new Exception("This state should never be reached, this means that we are ✓
processing in an "
                                + "area after an add command, but not before a remove command, check search ✓
bounds for inversion");

        // Loop over the next solid area
        for (int y = _markerScanCommandsY[mscIndexY].coordinate; y <= _markerScanCommandsY ✓
[mscIndexY + 1].coordinate; y++)
        {
            pPixelCurr = pPixel0 + image.Width * y + _markerScanCommandsX[mscIndexX].
coordinate;
            for (int x = _markerScanCommandsX[mscIndexX].coordinate; x <=
            _markerScanCommandsX[mscIndexX + 1].coordinate; x++, pPixelCurr++)
            {
                dirtyHSV = true;
                foreach (Marker marker in _scanMarkersX)
                {
                    // Get the pixel color in HSV
                    if (dirtyHSV)
                    {
                        // TODO: Remove indirection
                        hsv = RGB.ConvertToHSV(new RGB(*pPixelCurr));
                        dirtyHSV = false;
                    }

                    // Draw the search bounds
                    if (marker.Highlight && marker.prevData.Present)
                    {
                        if (x == marker.searchMinX.coordinate || x == marker.searchMaxX. ✓
coordinate ||
                            y == marker.searchMinY.coordinate || y == marker.searchMaxY. ✓
coordinate)
                            *pPixelCurr = marker.RepresentativeColor.ToArgb();
                    }

                    // Align the color with its appropriate bins, get color key
                    key.Hsv = HSV.GetBinnedHSV(hsv, new HSV(marker.binsHue, marker.binsSat, ✓
marker.binsVal));
                    if (marker.hsvFreq.Contains(key) && (int)marker.hsvFreq[key] > marker. ✓
Threshold)
                    {
                        marker.currData.Area++;
                        marker.currData.X += x;
                        marker.currData.Y += y;
                        if (y < marker.currData.top) marker.currData.top = y;
                        if (y > marker.currData.bottom) marker.currData.bottom = y;
                        if (x < marker.currData.left) marker.currData.left = x;
                        if (x > marker.currData.right) marker.currData.right = x;

                        // TODO: Better highlighting
                        if (marker.Highlight)
                        {
                            *pPixelCurr = marker.RepresentativeColor.ToArgb();

```

[illegible]

Works Cited

Anton Mikhailov. (2009-08-31) (Flash Video). PlayStation Motion Controller Interview Part 2. [podcast]. Foster City, California: Sony Computer Entertainment America. <<http://www.viddler.com/explore/sceablog/videos/546/>>.

"Autopsy Microsoft Kinect." *MaximumPC* Vol16.3 (March 2011): 61.

Burger, Wilhelm, Mark J. Burge. *Principles of Digital Image Processing*. London: Springer-Verlag, 2009 Print.

"Face Recognition Homepage." *Face-Rec*. 17 April 2011. Web. October 2010. <<http://www.face-rec.org/>>.

Forbes, Ron, Arjun Dayal. "How You Become the Controller." *Xbox Engineering*, Dec 29 2010. Web. Jan 2011. <<http://www.xbox.com/en-US/Live/EngineeringBlog/122910-HowYouBecometheController>>.

"Inside a Wii Remote" *How Stuff Works Express*. Web. February 2010. <<http://express.howstuffworks.com/wii-controller-autopsy.htm>>.

"QWERTY Keyboard" *The Great Idea Finder*. April 26, 2007. Web. June 2010. <<http://www.ideafinder.com/history/inventions/qwerty.htm>>.